

THE TALE OF LINUX SERVICES

A FRIENDLY ADVENTURE



Prepared By: Hosni Zaaraoui

"Your journey did not end there, dear traveler. Oh no, this was only the beginning!" With newfound confidence in managing Linux services, You sets out once more, eager to uncover deeper mysteries hidden within the kingdom. More secrets awaited discovery, more tools begged to be mastered, and more challenges stood in the way.

And so, the adventure continued...

Chapter 5: The Hidden World of Service Dependencies

One day, while tinkering with services, You notices something peculiar. Each time an attempt was made to start a service, another service would also start—or, strangely, refuse to run at all. This mystery demands answers.



Determined to uncover the truth, You travels deep into the `/etc/systemd/` directory, where the legendary Dependency Oracle resides. Within a dimly lit chamber filled with ancient scripts and glowing runes, the Oracle reveals a powerful secret:

"Services do not exist in isolation. Some depend on others to function. A web server, for instance, may require a database to be available before it can start. These relationships are encoded within the fabric of service unit files."

The Oracle points toward an enchanted terminal and instructs You to run:

```
systemctl list-dependencies nginx
```

As the command executes, a tree of interconnected services appears on the screen, revealing the hidden bonds between them.

"If You wishes to ensure a service starts only after its dependencies are ready," the Oracle continues, "use the After= directive in its unit file. And if ever You seeks the reason behind a service's failure, consult its status with this spell:"

```
systemctl status nginx
```

A sense of understanding dawns upon You. Dependencies are not mere obstacles—they are the glue binding the system together. With this knowledge, You leaves the Oracle's domain, prepared to wield this newfound power.

Chapter 6: The Quest for Resource Optimization

As the journey continues, You encounters a new challenge: certain services are consuming an excessive amount of resources, slowing down the system. CPU cycles vanish into thin air, memory drains like water through a sieve. The kingdom demands efficiency!



In search of wisdom, You ventures into the `/proc/` directory, where the enigmatic Resource Wizard dwells. Cloaked in robes woven from data streams, the Wizard shares a crucial lesson:

"To understand which services are consuming resources, consult the tools of monitoring."

```
top
```

```
htop
```

The terminal displays a lively battlefield of running processes, showing CPU, memory, and disk usage in real-time.

"But knowing is only half the battle," the Wizard continues. *"To impose limits upon a greedy service, modify its systemd unit file with these powerful constraints:"*

```
[Service]
CPUQuota=50%
MemoryMax=512M
```

With a flick of the wrist, the Wizard demonstrates the final step:

```
sudo systemctl daemon-reload

sudo systemctl restart myservice
```

Marveling at this newfound mastery, You realizes that controlling resources means controlling stability. The Wizard smiles and speaks a final warning:

"With great power comes great responsibility. Use it wisely."

Chapter 7: The Battle of the Containers

One day, a new force arrives at the gates of the Linux kingdom. Containers have begun to reshape the landscape, promising lightweight, isolated environments. Docker, the leader of this revolution, intrigues You. But how do containers fit within the world of Linux services?



Seeking answers, You travels to the Floating Islands of Containers, where the Container Sage awaits. The Sage explains:

"Containers are like miniature worlds of their own, running isolated services within. However, even they must be governed by systemd when integrated into a larger system."

To demonstrate, the Sage hands You a sacred scroll—a systemd unit file for managing a Docker container:

```
[Unit]
```

```
Description=My Awesome Container
```

```
After=docker.service
```

```
[Service]
```

```
ExecStart=/usr/bin/docker start -a my_container
```

```
ExecStop=/usr/bin/docker stop my_container
```

```
Restart=always
```

```
[Install]
```

```
WantedBy=multi-user.target
```

By saving this file as `/etc/systemd/system/mycontainer.service`, enabling it, and starting it, You bridges the gap between traditional services and modern containerization:

```
sudo systemctl enable mycontainer
```

```
sudo systemctl start mycontainer
```

The Sage nods approvingly. "You is now ready to embrace the future."

Chapter 8: The Chronicles of Security

With great power comes great danger. One fateful night, an urgent message arrives—a service has been compromised! The threat is real, and it is time to secure the kingdom.

You rushes to the `/etc/security/` stronghold, home to the Security Guardian. Clad in armor forged from firewalls and encryption, the Guardian shares the three golden rules of service security:

1. Run services with the least privileges possible. Modify unit files to specify a non-root user:

```
[Service]
User=www-data
Group=www-data
```

2. Fortify the gates with a firewall. Block unnecessary ports using:

```
sudo ufw deny 23 # Block Telnet (why are You even using
this?!)
sudo ufw allow 22/tcp # Allow SSH
```

3. Use Mandatory Access Control (MAC) frameworks like SELinux or AppArmor. Check active policies with:

```
sudo getenforce # For SELinux
```

```
sudo aa-status # For AppArmor
```

Finally, the Guardian hands You a final tool to reveal hidden threats:

```
ps aux | grep nginx
```

"Always know which user a service is running as," the Guardian warns. *"Never trust the unknown."*

With a renewed commitment to security, You vows to never leave a service vulnerable again.

Epilogue: The Never-Ending Adventure

The tale of Linux services is a never-ending saga, written by countless adventurers like You.

Before parting, the Oracle, the Wizard, the Sage, and the Guardian leave You with three eternal truths:

Stay curious.

Experiment fearlessly.

Share Your knowledge.

And so, dear traveler, the adventure continues...